

MySQL-Datenbankeinbindung mit Triggern und Stored Procedures

Ziel der Datenbankeinbindung

OpenHD FlightLog Studio speichert importierte MAVLink-Logdateien in einer lokalen MySQL-Datenbank. Die Anwendung soll dabei nicht die komplette Datenlogik im C#-Code verstecken. Stattdessen liegen zentrale Schreibregeln direkt in MySQL:

- Tabellen speichern Logs, MAVLink-Nachrichten, Felder, Definitionen und manuelle Variablen.
- Stored Procedures kapseln Insert-, Update- und Upsert-Vorgaenge.
- Trigger setzen automatisch abgeleitete Werte und halten Zaehler aktuell.
- Foreign Keys loeschen Detaildaten automatisch, wenn ein Log geloescht wird.

Der C#-Code bleibt dadurch hauptsaechlich fuer UI, Dateiimport, MAVLink-Parsing und das Aufrufen der Datenbankroutinen verantwortlich.

Verbindung zwischen Tool und MySQL

Die Verbindung wird in `FlightLogDatabase` aufgebaut. Die App nutzt `MySqlConnection` und liest die Zugangsdaten aus Umgebungsvariablen. Wenn keine Variablen gesetzt sind, nutzt sie lokale Standardwerte.

```
private static MySqlConnectionStringBuilder CreateConnectionStringBuilder()
{
    return new MySqlConnectionStringBuilder
    {
        Server = Environment.GetEnvironmentVariable("OPENHD_MYSQL_HOST") ?? "127.0.0.1",
        Port = uint.TryParse(Environment.GetEnvironmentVariable("OPENHD_MYSQL_PORT"), out var
port) ? port : 13306,
        UserID = Environment.GetEnvironmentVariable("OPENHD_MYSQL_USER") ?? "root",
        Password = Environment.GetEnvironmentVariable("OPENHD_MYSQL_PASSWORD") ?? "openhhd",
        Database = Environment.GetEnvironmentVariable("OPENHD_MYSQL_DATABASE") ??
"openhhd_flightlog",
        CharacterSet = "utf8mb4",
        SslMode = MySqlSslMode.None
    };
}
```

Beim Start der Datenbankschicht passiert:

1. Verbindungskonfiguration wird erstellt.
2. MySQL wird geprüfert oder per Docker gestartet.
3. Die Datenbank wird angelegt, falls sie fehlt.
4. Tabellen, Indizes, Trigger und Stored Procedures werden angelegt.

```
public FlightLogDatabase(Action<DebugEventRecord>? debug = null)
{
    this.debug = debug;
    connectionStringBuilder = CreateConnectionStringBuilder();
    DatabasePath = $"{connectionStringBuilder.Server}:
{connectionStringBuilder.Port}/{connectionStringBuilder.Database}";
    MySqlServerManager.EnsureServerStarted(connectionStringBuilder, Log);
    EnsureDatabase();
    EnsureSchema();
}
```

Schema-Erzeugung

Die Methode `EnsureSchema()` ist idempotent. Sie darf also bei jedem Start laufen. Existierende Tabellen bleiben erhalten, fehlende Tabellen werden angelegt.

Beispiel fuer die Wurzeltabelle eines Imports:

```
CREATE TABLE IF NOT EXISTS log_files (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    file_name VARCHAR(512) NOT NULL,
    path TEXT NOT NULL,
    imported_at VARCHAR(64) NOT NULL,
    import_started_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    message_count INT NOT NULL
) ENGINE=InnoDB;
```

Die Tabelle `mavlink_messages` speichert jedes erkannte MAVLink-Paket:

```
CREATE TABLE IF NOT EXISTS mavlink_messages (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    log_file_id BIGINT NOT NULL,  
    message_type_id BIGINT NOT NULL,  
    packet_index INT NOT NULL,  
    packet_time_ms BIGINT NOT NULL DEFAULT 0,  
    packet_timestamp VARCHAR(64) NOT NULL DEFAULT '',  
    byte_offset INT NOT NULL,  
    mavlink_version INT NOT NULL,  
    sequence INT NOT NULL,  
    system_id INT NOT NULL,  
    component_id INT NOT NULL,  
    route VARCHAR(255) NOT NULL DEFAULT '',  
    payload_length INT NOT NULL,  
    checksum VARCHAR(32) NOT NULL,  
    raw_packet_hex LONGTEXT NOT NULL,  
    FOREIGN KEY (log_file_id) REFERENCES log_files(id) ON DELETE CASCADE,  
    FOREIGN KEY (message_type_id) REFERENCES message_types(id)  
) ENGINE=InnoDB;
```

Wichtig ist `ON DELETE CASCADE` : Wenn ein Log gelöscht wird, löscht MySQL die zugehörigen Messages automatisch. Über die nächste Beziehung werden auch die Felder gelöscht.

Datenfluss beim Import

Der Import startet im `FlightLogImportService` .

```

public async Task<ImportResult> ImportLogAsync(string path)
{
    var bytes = await File.ReadAllBytesAsync(path);
    var frames = MavlinkParser.Parse(bytes);
    var fieldDefinitions = database.GetFieldDefinitionsByMessageId();
    var messageDefinitions = database.GetDefinitions().ToDictionary(definition =>
definition.MessageId);
    var packetTimings = OLogDebugSidecar.LoadPacketTimings(path);

    var importedMessages = new List<ImportedMavlinkMessage>();
    foreach (var frame in frames)
    {
        fieldDefinitions.TryGetValue(frame.MessageId, out var fields);
        messageDefinitions.TryGetValue(frame.MessageId, out var messageDefinition);
        packetTimings.TryGetValue(frame.PacketIndex, out var timing);

        importedMessages.Add(new ImportedMavlinkMessage(
            frame,
            timing,
            messageDefinition?.Name ?? MavlinkMessageDecoder.GetMessageName(frame.MessageId),
            messageDefinition?.Dialect ?? "",
            DynamicMavlinkDecoder.Decode(frame, fields ?? [])));
    }

    return database.SaveImportedLog(path, importedMessages);
}

```

Der Service parst also die Datei und dekodiert die MAVLink-Felder. Die eigentliche Speicherung laeuft danach ueber MySQL-Prozeduren.

Importtransaktion in C#

Alle Daten eines Imports werden in einer Transaktion geschrieben. Wenn ein Fehler auftritt, wird der Import nicht halb gespeichert.

```

public ImportResult SaveImportedLog(string path, IReadOnlyList<ImportedMavlinkMessage> messages)
{
    using var connection = OpenConnection();
    using var transaction = connection.BeginTransaction();

    var importStarted = DateTimeOffset.Now;
    var logId = InsertLogFile(connection, transaction, path, importStarted);

    foreach (var message in messages)
    {
        var frame = message.Frame;
        var messageId = EnsureMessageType(
            connection,
            transaction,
            frame.MessageId,
            message.MessageName,
            message.Dialect);

        var messageId = InsertMessage(connection, transaction, logId, messageId, message);

        foreach (var field in message.Fields)
        {
            InsertField(connection, transaction, messageId, field);
        }
    }

    transaction.Commit();
    return new ImportResult(logId, messages.Count, DatabasePath);
}

```

Man sieht hier: C# ruft nur noch Hilfsmethoden auf. Diese Hilfsmethoden fuehren keine grossen `INSERT` -Bloেকে mehr aus, sondern rufen Stored Procedures.

Stored Procedures

Stored Procedures sind gespeicherte SQL-Programme in MySQL. Sie enthalten die zentrale Schreiblogik und koennen von der Anwendung mit `CALL` ausgefuehrt werden.

`sp_create_log_file`

Diese Prozedur legt einen neuen Log-Eintrag an. Der Dateiname wird nicht im C#- Code gesetzt, sondern spaeter durch einen Trigger aus dem Pfad berechnet.

```

CREATE PROCEDURE sp_create_log_file(
    IN p_path TEXT,
    IN p_imported_at VARCHAR(64)
)
BEGIN
    INSERT INTO log_files (file_name, path, imported_at, message_count)
    VALUES ('', p_path, p_imported_at, 0);

    SELECT LAST_INSERT_ID() AS id;
END

```

C# ruft die Prozedur so auf:

```

command.CommandText = "CALL sp_create_log_file(@path, @importedAt);";
command.Parameters.AddWithValue("@path", path);
command.Parameters.AddWithValue("@importedAt", importedAt.ToString("yyyy-MM-dd HH:mm:ss zzz"));
var logId = Convert.ToInt64(command.ExecuteScalar());

```

sp_ensure_message_type

Diese Prozedur stellt sicher, dass eine MAVLink-Message-ID in `message_types` existiert. Wenn sie schon existiert, wird sie aktualisiert. Das ist ein Upsert.

```

CREATE PROCEDURE sp_ensure_message_type(
    IN p_message_id INT,
    IN p_name VARCHAR(255),
    IN p_dialect VARCHAR(255)
)
BEGIN
    INSERT INTO message_types (message_id, name, dialect)
    VALUES (p_message_id, TRIM(p_name), TRIM(COALESCE(p_dialect, '')))
    ON DUPLICATE KEY UPDATE
        id = LAST_INSERT_ID(id),
        name = VALUES(name),
        dialect = VALUES(dialect);

    SELECT LAST_INSERT_ID() AS id;
END

```

Der Vorteil: Die Anwendung muss nicht vorher prüfen, ob der Typ schon vorhanden ist. MySQL entscheidet das über den Unique Key auf `message_id`.

sp_insert_mavlink_message

Diese Prozedur speichert ein MAVLink-Paket. Route und Fallback-Zeitwerte werden nicht von C# gesetzt, sondern durch Trigger ergaenzt.

```
CREATE PROCEDURE sp_insert_mavlink_message(  
    IN p_log_file_id BIGINT,  
    IN p_message_type_id BIGINT,  
    IN p_packet_index INT,  
    IN p_packet_time_ms BIGINT,  
    IN p_packet_timestamp VARCHAR(64),  
    IN p_byte_offset INT,  
    IN p_mavlink_version INT,  
    IN p_sequence INT,  
    IN p_system_id INT,  
    IN p_component_id INT,  
    IN p_payload_length INT,  
    IN p_checksum VARCHAR(32),  
    IN p_raw_packet_hex LONGTEXT  
)  
BEGIN  
    INSERT INTO mavlink_messages  
        (log_file_id, message_type_id, packet_index, packet_time_ms, packet_timestamp,  
         byte_offset, mavlink_version, sequence, system_id, component_id,  
         route, payload_length, checksum, raw_packet_hex)  
    VALUES  
        (p_log_file_id, p_message_type_id, p_packet_index, COALESCE(p_packet_time_ms, 0),  
        COALESCE(p_packet_timestamp, ''),  
         p_byte_offset, p_mavlink_version, p_sequence, p_system_id, p_component_id,  
         '', p_payload_length, p_checksum, p_raw_packet_hex);  
  
    SELECT LAST_INSERT_ID() AS id;  
END
```

sp_insert_message_field

Diese Prozedur speichert ein dekodiertes Feld einer Message.

```
CREATE PROCEDURE sp_insert_message_field(  
    IN p_message_id BIGINT,  
    IN p_field_name VARCHAR(255),  
    IN p_value_text TEXT,  
    IN p_numeric_value DOUBLE,  
    IN p_unit VARCHAR(64)  
)  
BEGIN  
    INSERT INTO message_fields (message_id, field_name, value_text, numeric_value, unit)  
    VALUES (p_message_id, p_field_name, p_value_text, p_numeric_value, COALESCE(p_unit, ''));  
END
```

Wenn `numeric_value` leer ist, aber `value_text` eine Zahl enthaelt, setzt ein Trigger den Zahlenwert automatisch.

Weitere Prozeduren

Neben dem Import gibt es Prozeduren fuer Definitionen und manuelle Variablen:

```
CALL sp_upsert_message_definition(...);  
CALL sp_delete_definition_fields(...);  
CALL sp_insert_field_definition(...);  
CALL sp_update_message_definition(...);  
CALL sp_update_field_definition(...);  
CALL sp_update_message_field(...);  
CALL sp_save_user_variable(...);
```

Beispiel fuer manuelle Variablen:

```
CREATE PROCEDURE sp_save_user_variable(  
    IN p_id BIGINT,  
    IN p_name VARCHAR(255),  
    IN p_value_text TEXT,  
    IN p_data_type VARCHAR(64),  
    IN p_notes TEXT  
)  
BEGIN  
    IF p_id = 0 THEN  
        INSERT INTO user_variables (name, value_text, data_type, notes)  
        VALUES (TRIM(p_name), TRIM(p_value_text), COALESCE(NULLIF(TRIM(p_data_type), ''), 'text'),  
        TRIM(COALESCE(p_notes, '')));  
  
        SELECT LAST_INSERT_ID() AS id;  
    ELSE  
        UPDATE user_variables  
        SET name = TRIM(p_name),  
            value_text = TRIM(p_value_text),  
            data_type = COALESCE(NULLIF(TRIM(p_data_type), ''), 'text'),  
            notes = TRIM(COALESCE(p_notes, ''))  
        WHERE id = p_id;  
  
        SELECT p_id AS id;  
    END IF;  
END
```

Trigger

Trigger sind automatische Aktionen, die MySQL bei bestimmten Tabellenereignissen ausführt. Sie laufen ohne extra Aufruf aus C#.

trg_log_files_before_insert

Dieser Trigger läuft vor jedem Insert in `log_files`.

Aufgaben:

- Dateiname aus dem Pfad ableiten.
- Importzeit setzen, falls sie leer ist.
- Negative oder leere Message-Counts verhindern.

```

CREATE TRIGGER trg_log_files_before_insert
BEFORE INSERT ON log_files
FOR EACH ROW
BEGIN
    IF NEW.file_name IS NULL OR TRIM(NEW.file_name) = '' THEN
        SET NEW.file_name = SUBSTRING_INDEX(REPLACE(NEW.path, '\\', '/'), '/', -1);
    END IF;

    IF NEW.imported_at IS NULL OR TRIM(NEW.imported_at) = '' THEN
        SET NEW.imported_at = DATE_FORMAT(NOW(3), '%Y-%m-%d %H:%i:%s');
    END IF;

    IF NEW.message_count IS NULL OR NEW.message_count < 0 THEN
        SET NEW.message_count = 0;
    END IF;
END

```

Beispiel:

```

CALL sp_create_log_file('C:\\Logs\\sample_openhd.oLog', '2026-06-19 10:30:00 +02:00');

SELECT file_name, path
FROM log_files
ORDER BY id DESC
LIMIT 1;

```

Erwartetes Ergebnis:

file_name	path
sample_openhd.oLog	C:\Logs\sample_openhd.oLog

trg_mavlink_messages_before_insert

Dieser Trigger laeuft vor jedem Insert in `mavlink_messages`.

Aufgaben:

- Route automatisch aus `system_id` ableiten.
- `packet_time_ms` als Fallback auf `packet_index` setzen.
- `packet_timestamp` als Fallback aus Importzeit plus Paketindex setzen.

```

CREATE TRIGGER trg_mavlink_messages_before_insert
BEFORE INSERT ON mavlink_messages
FOR EACH ROW
BEGIN
    IF NEW.route IS NULL OR TRIM(NEW.route) = '' THEN
        SET NEW.route = CASE NEW.system_id
            WHEN 100 THEN 'OpenHD Ground'
            WHEN 101 THEN 'OpenHD Air'
            WHEN 255 THEN 'QOpenHD'
            WHEN 1 THEN 'Flight Controller'
            WHEN 0 THEN 'Flight Controller'
            ELSE CONCAT('System ', NEW.system_id)
        END;
    END IF;

    IF NEW.packet_time_ms IS NULL OR NEW.packet_time_ms = 0 THEN
        SET NEW.packet_time_ms = NEW.packet_index;
    END IF;

    IF NEW.packet_timestamp IS NULL OR TRIM(NEW.packet_timestamp) = '' THEN
        SET NEW.packet_timestamp = (
            SELECT DATE_FORMAT(DATE_ADD(import_started_at, INTERVAL (NEW.packet_index * 1000)
MICROSECOND), '%Y-%m-%d %H:%i:%s.%f')
            FROM log_files
            WHERE id = NEW.log_file_id
        );
    END IF;
END

```

Beispiel fuer die Route:

```

-- system_id 100 wird automatisch zu "OpenHD Ground"
CALL sp_insert_mavlink_message(
    1, 1, 42, NULL, NULL,
    128, 2, 7, 100, 1,
    12, '0x1234', 'FD0C0000006401000000000000000000000000'
);

SELECT packet_index, system_id, route, packet_time_ms
FROM mavlink_messages
WHERE packet_index = 42;

```

Erwartetes Ergebnis:

packet_index	system_id	route	packet_time_ms
42	100	OpenHD Ground	42

trg_mavlink_messages_after_insert

Dieser Trigger laeuft nach jedem Insert in `mavlink_messages`.

Aufgabe:

- `log_files.message_count` automatisch neu berechnen.

```
CREATE TRIGGER trg_mavlink_messages_after_insert
AFTER INSERT ON mavlink_messages
FOR EACH ROW
BEGIN
    UPDATE log_files
    SET message_count = (
        SELECT COUNT(*)
        FROM mavlink_messages
        WHERE log_file_id = NEW.log_file_id
    )
    WHERE id = NEW.log_file_id;
END
```

Dadurch muss C# die Anzahl der Messages nicht selbst mitzaehlen.

trg_mavlink_messages_after_delete

Dieser Trigger laeuft nach dem Loeschen einer Message.

```
CREATE TRIGGER trg_mavlink_messages_after_delete
AFTER DELETE ON mavlink_messages
FOR EACH ROW
BEGIN
    UPDATE log_files
    SET message_count = (
        SELECT COUNT(*)
        FROM mavlink_messages
        WHERE log_file_id = OLD.log_file_id
    )
    WHERE id = OLD.log_file_id;
END
```

Wenn Messages entfernt werden, bleibt der Zaehler in `log_files` korrekt.

trg_mavlink_messages_after_update

Dieser Trigger wird relevant, wenn eine Message theoretisch einem anderen Log zugeordnet wird.

```
CREATE TRIGGER trg_mavlink_messages_after_update
AFTER UPDATE ON mavlink_messages
FOR EACH ROW
BEGIN
    IF OLD.log_file_id <> NEW.log_file_id THEN
        UPDATE log_files
        SET message_count = (
            SELECT COUNT(*)
            FROM mavlink_messages
            WHERE log_file_id = OLD.log_file_id
        )
        WHERE id = OLD.log_file_id;

        UPDATE log_files
        SET message_count = (
            SELECT COUNT(*)
            FROM mavlink_messages
            WHERE log_file_id = NEW.log_file_id
        )
        WHERE id = NEW.log_file_id;
    END IF;
END
```

trg_message_fields_before_insert

Dieser Trigger laeuft vor jedem Insert in `message_fields`.

Aufgaben:

- Feldname, Textwert und Einheit trimmen.
- Zahlen automatisch erkennen.

```
CREATE TRIGGER trg_message_fields_before_insert
BEFORE INSERT ON message_fields
FOR EACH ROW
BEGIN
    SET NEW.field_name = TRIM(NEW.field_name);
    SET NEW.value_text = TRIM(NEW.value_text);
    SET NEW.unit = TRIM(COALESCE(NEW.unit, ''));

    IF NEW.numeric_value IS NULL AND NEW.value_text REGEXP '^-[0-9]+(\\.[0-9]+)?$' THEN
        SET NEW.numeric_value = CAST(NEW.value_text AS DECIMAL(30,10));
    END IF;
END
```

Beispiel:

```
CALL sp_insert_message_field(1, ' voltage_battery ', ' 11.7 ', NULL, ' V ');

SELECT field_name, value_text, numeric_value, unit
FROM message_fields
ORDER BY id DESC
LIMIT 1;
```

Erwartetes Ergebnis:

field_name	value_text	numeric_value	unit
voltage_battery	11.7	11.7	V

trg_message_fields_before_update

Dieser Trigger verwendet dieselbe Logik beim Bearbeiten vorhandener Felder.

```
CREATE TRIGGER trg_message_fields_before_update
BEFORE UPDATE ON message_fields
FOR EACH ROW
BEGIN
    SET NEW.field_name = TRIM(NEW.field_name);
    SET NEW.value_text = TRIM(NEW.value_text);
    SET NEW.unit = TRIM(COALESCE(NEW.unit, ''));

    IF NEW.numeric_value IS NULL AND NEW.value_text REGEXP '^-[0-9]+(\\.[0-9]+)?$' THEN
        SET NEW.numeric_value = CAST(NEW.value_text AS DECIMAL(30,10));
    END IF;
END
```

Warum Trigger und Prozeduren sinnvoll sind

Ohne Trigger und Prozeduren muesste die Anwendung viele Regeln selbst ausfuehren:

```
var route = systemId switch
{
    100 => "OpenHD Ground",
    101 => "OpenHD Air",
    255 => "QOpenHD",
    1 or 0 => "Flight Controller",
    _ => $"System {systemId}"
};
```

Diese Route-Logik liegt jetzt in MySQL. Das hat Vorteile:

- Die Regel gilt immer, egal ob Daten aus der App oder direkt per SQL eingefuegt werden.
- Der C#-Code wird kleiner und klarer.
- Die Datenbank schuetzt ihre eigene Datenqualitaet.
- Wiederholte Schreibablaeufe werden zentral gepflegt.

Zusammenspiel von C# und MySQL

Die Arbeit ist aufgeteilt:

Bereich	Verantwortung
C# UI	Anzeigen, Auswaehlen, Buttons, Tabellenansichten
C# Importservice	Datei lesen, MAVLink-Pakete parsen, Felder dekodieren
MySQL Stored Procedures	Zentrale Insert-, Update- und Upsert-Vorgaenge
MySQL Trigger	Automatische Ableitungen und Konsistenzregeln
MySQL Foreign Keys	Automatisches Loeschen abhaengiger Daten

Ein typischer Import sieht deshalb so aus:

```
Benutzer waehlt .oLog-Datei
    |
    v
C# liest Datei und parst MAVLink-Frames
    |
    v
C# ruft Stored Procedures mit CALL auf
    |
    v
MySQL schreibt Tabellen
    |
    v
Trigger ergaenzen Route, Zeitstempel, Dateiname, Message-Count
    |
    v
UI liest die gespeicherten Daten per SELECT wieder aus
```

Beispiel: kompletter Mini-Import in SQL

Das folgende Beispiel zeigt vereinfacht, wie die App intern mit MySQL interagiert.

```
CALL sp_create_log_file('C:\\Logs\\demo.oLog', '2026-06-19 12:00:00 +02:00');  
SET @log_id = LAST_INSERT_ID();  
  
CALL sp_ensure_message_type(30, 'ATTITUDE', 'common');  
SET @type_id = LAST_INSERT_ID();  
  
CALL sp_insert_mavlink_message(  
    @log_id,  
    @type_id,  
    1,  
    NULL,  
    NULL,  
    0,  
    2,  
    10,  
    1,  
    1,  
    28,  
    '0xABCD',  
    'FD1C0000A01011E000000000000000000000000000000000000000000000000000000000000000000000000000000000'  
);  
SET @message_id = LAST_INSERT_ID();  
  
CALL sp_insert_message_field(@message_id, 'roll', '0.12', NULL, 'rad');  
CALL sp_insert_message_field(@message_id, 'pitch', '-0.04', NULL, 'rad');  
CALL sp_insert_message_field(@message_id, 'yaw', '1.57', NULL, 'rad');
```

Nach diesen Aufrufen haben Trigger automatisch:

- `log_files.file_name` auf `demo.oLog` gesetzt.
- `mavlink_messages.route` auf `Flight Controller` gesetzt.
- `mavlink_messages.packet_time_ms` auf `1` gesetzt.
- `mavlink_messages.packet_timestamp` aus `import_started_at` berechnet.
- `log_files.message_count` auf `1` gesetzt.
- `message_fields.numeric` value aus den Textwerten berechnet.

Kontrollabfrage:

```
SELECT
  l.file_name,
  l.message_count,
  m.packet_index,
  m.route,
  f.field_name,
  f.value_text,
  f.numeric_value,
  f.unit
FROM log_files l
JOIN mavlink_messages m ON m.log_file_id = l.id
JOIN message_fields f ON f.message_id = m.id
WHERE l.id = @log_id
ORDER BY m.packet_index, f.field_name;
```

Fazit

Die App nutzt MySQL nicht nur als einfache Ablage. Die Datenbank enthaelt eigene Logik:

- Stored Procedures bilden die offiziellen Schreibschnittstellen.
- Trigger fuehren automatische Regeln aus.
- Foreign Keys sichern die Beziehungen zwischen Logs, Messages und Feldern.

Damit ist die Datenbankeinbindung transparenter und entspricht besser der Anforderung, wichtige Datenlogik nicht nur im Anwendungscode zu verstecken.